

Academy Backend/Frontend/QE Virtual MTY

Proyecto: Sistema Básico de Gestión - Cine San Pedro

Desarrollador: Sergio Servando Prado Lozano

Encargado: Miguel Ángel Rugerio Flores

Lugar: San Pedro de las Colonias Coahuila

Fecha: 15/08/2026

1.- Introducción Proyecto

El presente proyecto consiste en el desarrollo de un Sistema de Gestión Básico para el Cine de San Pedro. El objetivo principal de este desarrollo es llevar a la práctica los conceptos fundamentales de la Programación Orientada a Objetos (POO) mediante la simulación de una taquilla real. A través de este sistema, se controlan las salas, la cartelera de películas y la venta de boletos con diferentes categorías y precios, mostrando el dominio de herramientas como la herencia, el polimorfismo y la encapsulación y más conceptos vistos dentro de la primera semana de la academia.

2.- Versión 0 (cinesanpedrocoahuila_v0): Arquitectura Base

En esta primera etapa o borrador principal (paquete cinesanpedrocoahuila_v0), se realizó la arquitectura fundamental del sistema. El enfoque principal fue diseñar las clases, establecer sus relaciones y asegurar que la lógica del sistema funcionará correctamente mediante diferentes pruebas.

2.1.- Herencia y Clases Abstractas

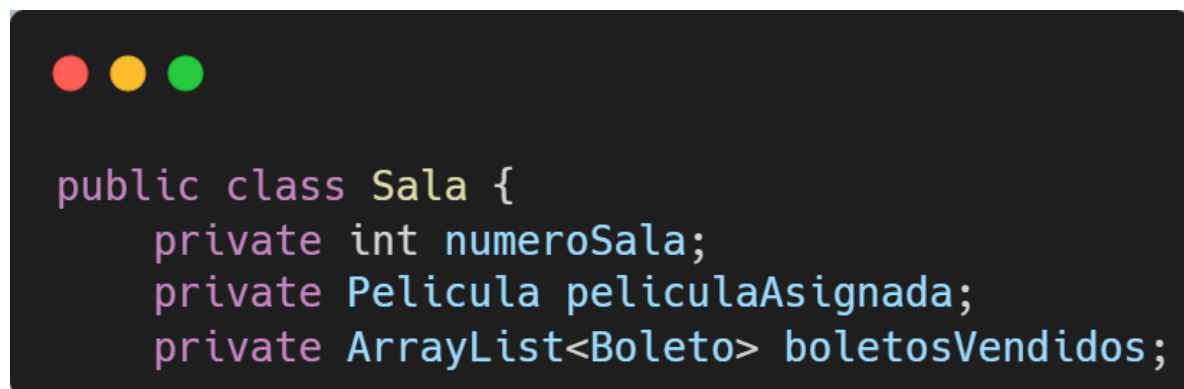
Para manejar los tipos de entrada, se definió una clase abstracta padre llamada Boleto. Esta clase no puede ser instanciada por sí misma y obliga a sus clases hijas a implementar el método calcularPrecio(). A partir de esta, se crearon dos clases hijas utilizando la herencia (extends): BoletoBasico y BoletoPremium. Cada una define su propio precio utilizando polimorfismo. Adicionalmente, el BoletoPremium incluye un método exclusivo llamado pedirComidaAsiento().

2.2.- Encapsulación e Inmutabilidad

Para proteger la información crítica, clases como Cine y Pelicula utilizan el modificador `private` final en sus atributos. Esto garantiza la inmutabilidad de los datos; una vez que una película recibe su título y `duracionMins` a través del constructor, estos no pueden ser modificados accidentalmente. El acceso a estos datos se realiza estrictamente a través de métodos `Getters`.

2.3.- Composición (Has-A) y Colecciones (Generics)

La clase `Sala` actúa como el núcleo de la lógica de cada función. Esta clase tiene una relación de composición, ya que agrupa una `Pelicula` específica y una lista de ventas. Para administrar las ventas, se implementó la herramienta `ArrayList` utilizando `Generics` (`<Boleto>`) para garantizar que la lista solo reciba objetos de este tipo.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in a light-colored font and shows the definition of a Java class named 'Sala'. The class has three private attributes: 'numeroSala' of type 'int', 'peliculaAsignada' of type 'Pelicula', and 'boletosVendidos' of type 'ArrayList<Boleto>'.

```
public class Sala {  
    private int numeroSala;  
    private Pelicula peliculaAsignada;  
    private ArrayList<Boleto> boletosVendidos;
```

Código 1.- Imagen dónde se muestra el implemento de la herramienta.

2.4.- Pruebas de Ejecución y Polimorfismo

Finalmente, la clase `Principal` contiene el método estático `main` que orquesta el sistema. En esta versión, las ventas se registraron de manera directa en el código. Al generar los reportes de ventas, el

sistema recorre los boletos y aplica un Casteo (Cast) explícito cuando detecta una instancia de BoletoPremium para poder invocar su método especial:

```
for (Boleto b : salaActual.getBoletosVendidos()) {  
    System.out.println("El asiento " + b.getNumeroAsiennto() + " ---> "  
+ "Se cobró: $" + b.calcularPrecio());  
    if (b instanceof BoletoPremium) {  
        BoletoPremium premium = (BoletoPremium) b;  
        premium.pedirComidaAsiento();  
    }  
}
```

Código 2.- Aquí se aplica la herramienta de casteo para decir que este Boleto lo tomen como Boleto Premium.

3.- Versión 1 (cinesanpedrocoahuila_v1): Implementación Interactiva de Taquilla

En la segunda etapa del proyecto (paquete cinesanpedrocoahuila_v1), el sistema mejoró de una ejecución estática a una aplicación de consola completamente interactiva. El objetivo principal fue permitir que un usuario real operara la taquilla de manera continua, tomando decisiones en tiempo real sobre la venta de las entradas.

3.1.- Interacción con el Usuario (Scanner)

Para lograr la interactividad, se integró la clase Scanner (instanciada como teclas) para capturar la entrada de datos a través del teclado. El sistema ahora solicita de manera dinámica al usuario que ingrese variables como el número de sala (numeroSala), el tipo de boleto (tipoBoleto) y numero del asiento (asientoEleccion). Dependiendo de estas elecciones, el programa crea la instancia correspondiente mediante sentencias if/else.

3.2.- Control de Flujo con Ciclos (While)

Para simular un entorno real donde la taquilla atiende a múltiples clientes uno tras otro, se integró un ciclo while. Este ciclo está controlado por una variable de tipo boolean llamada taquillaActiva, la cual se inicializa en true. Mientras esta condición se mantenga verdadera, el menú principal de selección de películas se repetirá en la pantalla tras cada venta exitosa.

3.3.- Cierre de Taquilla

Se hizo una condición de salida (opción 0) en el menú principal para cerrar las operaciones. Al ingresar este valor numérico, el sistema cambia el estado de la variable taquillaActiva a false. Esto finaliza inmediatamente el ciclo while y desencadena la impresión automática del reporte final, el cual recorre las salas e imprime los cálculos totales mediante polimorfismo.

```

boolean taquillaActiva = true;
Scanner teclas = new Scanner(System.in);

while (taquillaActiva) {
    System.out.println("-----");
    System.out.println("Estas en la Taquilla");
    System.out.println("-----");

    System.out.println("¿Qué película deseas ver? Elige la película");
    System.out.println("1 - " + sala1.getPeliculaAsignada().getTitulo());
    System.out.println("2 - " + sala2.getPeliculaAsignada().getTitulo());
    System.out.println("3 - " + sala3.getPeliculaAsignada().getTitulo());
    System.out.println("0 - Para salir y ver estado del cine");

    int numeroSala = teclas.nextInt();
    teclas.nextLine();

    if (numeroSala == 0) {
        System.out.println("-----");
        System.out.println("Taquilla cerrada, aquí esta el reporte del día");
        taquillaActiva = false;
    }
    else if (numeroSala == 1 || numeroSala == 2 || numeroSala == 3) {
        System.out.println("¿Qué boleto quiere?");
        System.out.println("1- Basico en $50");
        System.out.println("2- Premium en $100");
        int tipoBoleto = teclas.nextInt();
        teclas.nextLine();

        System.out.println("¿Dónde se gustaría sentar?");
        System.out.println("Si es Basico: B1, B2 ...");
        System.out.println("Si es Premium: P1, P2 ...");
        String asientoEleccion = teclas.nextLine();

        if (numeroSala == 1) {
            if (tipoBoleto == 1) {
                sala1.venderBoleto(new BoletoBasico(asientoEleccion));
            }
            else {
                sala1.venderBoleto(new BoletoPremium(asientoEleccion));
            }
            System.out.println("-----");
            System.out.println("Boleto vendido para la Sala 1");
        }
        else if (numeroSala == 2) {
            if (tipoBoleto == 1) {
                sala2.venderBoleto(new BoletoBasico(asientoEleccion));
            }
            else {
                sala2.venderBoleto(new BoletoPremium(asientoEleccion));
            }
            System.out.println("-----");
            System.out.println("Boleto vendido para la Sala 2");
        }
        else if (numeroSala == 3) {
            if (tipoBoleto == 1) {
                sala3.venderBoleto(new BoletoBasico(asientoEleccion));
            }
            else {
                sala3.venderBoleto(new BoletoPremium(asientoEleccion));
            }
            System.out.println("-----");
            System.out.println("Boleto vendido para la Sala 3");
        }
    }
}
}

```

Código 3.- Imagen del código dónde se muestra de forma extendida el funcionamiento del while y scanner.

4. Conclusiones del Proyecto

El desarrollo de este sistema de gestión básico pusimos en práctica el paradigma de la Programación Orientada a Objetos se traduce en herramientas reales y funcionales. Al establecer primero una arquitectura o Versión 0 hecha con los conceptos como la abstracción, la herencia, la encapsulación y el polimorfismo, la escalabilidad del proyecto no fue forzado. El salto hacia la aplicación interactiva o Versión 1 fue un bastante bien porque se logró el comportamiento estricto de las salas y los boletos de una manera sencilla también se logró la lógica de interfaz por ejemplo la lectura de teclado y menús del main. El código resultó ser bueno y cuenta con bases excelentes para continuar implementando nuevos requerimientos, validaciones y patrones de diseño conforme se vayan planteando las modificaciones.